

Leonor Rebola

Estudante nº 53663

Curso: Inteligência Artificial e Ciência de Dados

UC: Interação com Modelos em Larga Escala

Repositório GitHub

Covilhã, Maio de 2026

TP 1: From Raw Detections to Real Intelligence

1 Introdução

O seguinte trabalho implementa um pipeline de análise de dados de uma loja de retalho que transforma eventos brutos de deteção por câmara em inteligência operacional útil para um gestor de loja. O sistema recebe um fluxo de eventos anónimos (sem ID) e produz automaticamente um briefing semanal com *insights* acionáveis, anomalias detetadas e recomendações concretas.

O trabalho é composto por quatro módulos independentes:

- `stitcher.py`: reconstrução das trajetórias individuais a partir de eventos anónimos
- `analytics.py`: cálculo de métricas operacionais
- `insights.py`: geração de *insights* com LLM
- `report.py`: geração do briefing semanal

Em termos de arquitetura, o Python é responsável por todo o cálculo numérico, sendo que os resultados são depois entregues à LLM já processados em formato JSON. Desta forma, a LLM limita-se a interpretar e a redigir, sem acesso direto aos dados em bruto. Em termos de limitações, o dataset apresenta imperfeições intrínsecas ao sistema de visão computacional, já que as estimativas de género têm uma taxa de erro de aproximadamente 8% e as de faixa etária de 12%. Adicionalmente, os sensores de entrada falham periodicamente, produzindo eventos de saída sem entrada correspondente. Estas limitações são documentadas ao longo do relatório e refletem-se nas métricas de qualidade do sistema.

2 Algoritmo de stitching

2.1 Abordagem greedy

Face ao volume de dados e à restrição de execução em menos de 5 minutos, decidi implementar um algoritmo greedy, em que os eventos são processados por ordem cronológica e, para cada novo evento, é tomada imediatamente a melhor decisão de associação disponível, sem revisão de decisões anteriores. Optei por esta abordagem, pois é computacionalmente mais leve.

Esta abordagem mantém em memória apenas as trajetórias que têm em conta pessoas que estão na loja naquele exato momento, o que faz com que a complexidade se mantenha próxima de $O(n)$.

O resultado foram 14.390 trajetórias reconstruídas num tempo de execução de, aproximadamente, 17 segundos.

2.2 Heurísticas de associação de trajetórias

Esta parte é feita por duas fases, sendo a primeira de eliminação de trajetórias inválidas e a segunda de classificação e pontuação das candidatas viáveis. Para a primeira fase, foram tidos em conta os seguintes filtros:

- **Sobreposição temporal:** a pessoa ainda está dentro de uma zona e o gap é inferior a 5 segundos, o que torna fisicamente impossível estar em dois sítios ao mesmo tempo.
- **Gap excessivo:** o tempo desde o último evento da trajetória excede 600 segundos, assumindo-se que a visita já terminou.
- **Implausibilidade física:** o tempo disponível é inferior a 60% do tempo mínimo de deslocação entre as duas zonas, calculado a partir do grafo de adjacências em `zones.json`.
- **Inconsistência demográfica acumulada:** a trajetória já acumulou dois ou mais erros demográficos e o novo evento diverge em ambos os atributos.

Na segunda fase, para ordenar as trajetórias sobranes, foi usado o seguinte cálculo

$$S = 0,5 \cdot gap + 0,4 \cdot attr + 0,1 \cdot adj \quad (1)$$

onde o gap temporal tem peso dominante por ser o critério fisicamente mais fiável, seguido dos atributos demográficos e da adjacência entre zonas.

O valor de `MAX_GAP_S` foi ajustado ao longo das experiências já que, com o valor inicialmente usado (300 segundos), o sistema produzia 10.983 trajetórias com 82,6% de cobertura e 92,2% de completude. Ao aumentar para 600 segundos, a cobertura subiu para 98,7% e o número de trajetórias para 14.390, mantendo a consistência em 100%. A escolha do valor final privilegia a cobertura, mesmo com uma pequena descida da completude.

Esta decisão justifica-se pelo facto do cálculo dos tempos de permanência (*dwell times*) ser mais importante do que garantir que as viagens começam e acabam nas portas da loja.

2.3 Trajetórias incompletas

As trajetórias incompletas são aquelas que não começam numa zona de entrada e são excluídas do cálculo do funil de clientes.

Quando um evento de entrada numa zona interior não encontra nenhuma trajetória compatível, é criada uma nova trajetória incompleta em vez de descartar o evento, privilegiando a cobertura, pois um evento descartado perde-se definitivamente, enquanto uma trajetória incompleta pode ainda receber eventos subsequentes à medida que a pessoa continua a ser detetada na loja..

2.4 Métricas de qualidade

Métrica	Valor	Interpretação
Consistência	100,0%	Nenhuma trajetória com sobreposição temporal
Cobertura	98,7%	Entradas associadas a alguma trajetória
Compleitude	87,2%	Trajetórias com entrada e saída em Z_E ou Z_CK
Gap mediano	9,0s	Plausível para loja de retalho (adjacências: 8–20s)
Gap p95	31,0s	95% dos gaps abaixo de 31 segundos

Tabela 1: Métricas de qualidade do stitching no dataset de treino.

A consistência de 100% é a métrica mais importante, pois garante que o algoritmo nunca coloca a mesma pessoa em dois sítios ao mesmo tempo.

2.5 Casos de falha e limitações

O principal caso de falha identificado corresponde a trajetórias compostas por uma única zona visitada (cerca de 47% do total, comprovado através de `check.py`). Estas derivam quase exclusivamente de eventos de saída sem o registo de entrada correspondente, devido a falhas de deteção dos sensores nas portas das zonas.

Foi também identificada uma falha residual (inferior a 0,1% do total) caracterizada por trajetórias excessivamente longas, resultantes da fusão indevida de eventos de pessoas diferentes com perfis demográficos semelhantes.

A implementação de uma regra de mitigação, com o corte artificial de trajetórias com base num limite máximo de zonas visitadas foi testada porém, esta abordagem produziu resultados globais piores, penalizando a cobertura de trajetórias longas e legítimas, pelo que a sua aplicação foi abandonada.

3 Pipeline analítico

O script `analytics.py` calcula cinco categorias de métricas: tráfego, métricas por zona, segmentação demográfica, funil de clientes e deteção de anomalias.

Todos os resultados são exportados para o ficheiro `metrics.json`, que serve de input para o modelo de linguagem na fase seguinte.

3.1 Funil de conversão e planta da Loja

A análise do funil revelou inicialmente que 30,8% dos visitantes chegaram à zona das caixas, mas apenas 11,5% passaram pelos corredores interiores. A justificação reside na planta física da loja, onde as portas de entrada estão imediatamente adjacentes às caixas, o que permite uma visita rápida, onde o cliente entra e pode sair sem passar pelos corredores.

Para lidar com isto e garantir métricas realistas, implementei um funil sequencial, contabilizando apenas as zonas visitadas antes do primeiro evento de caixa. Além disso, as trajetórias que não iniciam nas zonas de entrada (Z.E) foram excluídas deste cálculo.

3.2 Deteção de anomalias

A deteção de anomalias compara o sétimo dia com a média dos primeiros seis. Algo importante a garantir foi a ausência total de visitantes numa zona a uma determinada hora é também reconhecida como uma anomalia, fazendo com que situações como obstruções físicas são corretamente sinalizadas.

3.3 Ficheiro `metrics.json`

O formato do `metrics.json` foi desenhado como um sumário executivo estruturado. A decisão de incluir apenas métricas agregadas e o top 5 de anomalias ao invés de trajetórias individuais deveu-se à necessidade de otimizar a janela de contexto do LLM. Excluíram-se os dados brutos para prevenir alucinações aritméticas e garantir que o modelo se foca na interpretação estratégica.

4 Engenharia de prompts

Os scripts *insights.py* e *report.py* utilizam o modelo `llama3.1:8b` via Ollama, correndo localmente com `temperature=0.0` para garantir reprodutibilidade.

4.1 Comparação de estratégias

Foram testadas duas abordagens de *prompting*, sendo elas:

- **Estratégia A (Zero-Shot):** instrução direta com o *schema* de saída, sem exemplos de referência. O modelo demonstrou dificuldades em distinguir métricas triviais de *insights* estratégicos.
 - *Exemplo de falha:* "A zona Z_E1 é a mais visitada". Este dado é redundante, pois Z_E1 é a entrada principal da loja. O modelo falhou ao não sugerir ações para as zonas de produto onde o tráfego é convertido em vendas.
- **Estratégia B (Few-Shot):** inclusão de exemplos de "Bons vs. Maus Insights". Esta abordagem permitiu ao modelo compreender que um bom *insight* deve cruzar desvios estatísticos (Z-Scores) com recomendações operacionais.

Métrica	Estratégia A (Zero-Shot)	Estratégia B (Few-Shot)
Especificidade (Dados concretos)	0,57	0,67
Acionabilidade (Verbos de ação)	0,65	0,88
Score Final Combinado	0,74	0,85

Tabela 2: Comparação quantitativa da qualidade dos *outputs* do LLM

A superioridade da estratégia B é evidente, superando a A em todas as métricas, com destaque para a acionabilidade.

4.2 Iterações e refinamento do prompt

O processo de refinamento focou-se em reduzir a reprodução literal dos exemplos, onde o modelo tendia a copiar as zonas usadas nos exemplos. A solução passou por dois ajustes:

1. **Zonas fictícias:** substituição das zonas reais nos exemplos por identificadores como Z_X9, forçando o modelo a procurar as zonas relevantes nos dados reais da semana.
2. **Rigor categórico:** restrição a uma categoria única por *insight*, evitando respostas ambíguas como **tráfego/zona**.

5 Avaliação

5.1 Harness de avaliação

O módulo `evaluate.py` executa o pipeline completo de forma autónoma sobre qualquer dataset no formato correto, garantindo que a avaliação é independente de estado anterior e reproduzível através do comando

```
python evaluate.py --data events_validation.csv \ --output evaluation_report.json
```

5.2 Resultados

Métrica	Valor	Nota técnica
Consistência	100,0%	Sem violações temporais ou sobreposições físicas
Cobertura	100,0%	Entradas no <code>journeys.csv</code> / entradas totais no dataset
Compleitude	87,2%	Trajetoórias com validação de entrada (Z_E) e saída (Z_CK)
Anomalias detetadas	22	Desvios superiores a 2σ no 7º dia
Precisão numérica	95,2%	Números nos <i>insights</i> verificáveis e validados pelos dados
Taxa de alucinação	4,3%	Taxa residual de erro não verificável no relatório final
Tempo total	104,6s	Execução de todo o <i>pipeline</i> , incluindo inferência do LLM

Tabela 3: Resultados globais extraídos do *harness* de avaliação do dataset fornecido

5.3 Análise de falhas

O principal caso de falha prende-se com a elevada percentagem de trajetórias com uma única zona visitada (47% do total), originadas maioritariamente por falhas de registo dos sensores, devido à planta da loja, onde as caixas estão perto da saída, o que faz com que muitos clientes faça apenas um pagamento. Consequentemente, torna-se impossível para o algoritmo distinguir com certeza se uma trajetória curta na zona da caixa corresponde a uma visita rápida ou à visita de um cliente que percorreu a loja inteira sem ser detetado noutros corredores.

Um segundo caso de falha são os insights gerados pelo LLM com recomendações vagas.

5.4 Limitações do sistema

As limitações que mais destaque merecem são:

1. **Qualidade do stitching:** 47% das trajetórias têm uma única zona visitada, resultantes de falhas do sensor de entrada, devendo-se a um problema estrutural.
2. **Modelo LLM de pequena dimensão:** o `llama3.1:8b` produz ocasionalmente recomendações genéricas e tende a copiar os exemplos do few-shot. Um modelo de maior dimensão produziria insights mais sofisticados.

5.5 Cenários de teste com anomalias injetadas

Para validar o sistema de detecção de anomalias antes de receber o dataset de validação, foram injetadas três anomalias conhecidas no dataset original através do script `inject_anomalies.py`:

ID	Tipo	Descrição
ANO_001	Queda para zero	Remoção de 56 eventos em Z_S3 às 14h do dia 7 (simula encerramento temporário)
ANO_002	Pico de tráfego	Duplicação de 129 eventos em Z_N5 às 11h do dia 7 (simula evento promocional)
ANO_003	Queda parcial	Remoção de 648 eventos (80%) em Z_C2 às 17h do dia 7 (simula avaria de caixa)

Tabela 4: Anomalias injetadas no dataset de teste

O pipeline foi executado sobre o dataset modificado com os seguintes com consistência de 100%, cobertura de 100%, completude de 87,3% e 18 anomalias detectadas, mas nenhuma das três anomalias injetadas apareceu no top 5 de anomalias reportadas. Esta limitação revela que o sistema é sensível a anomalias de grande magnitude, mas pode não detectar desvios moderados em zonas com elevada variabilidade natural de tráfego.

6 Conclusão

O desenvolvimento deste pipeline de análise de retalho permitiu transformar um volume massivo de dados com ruído em inteligência de negócio acionável, revelando aprendizagens importantes sobre algoritmos heurísticos e a integração de modelos de linguagem locais.

6.1 O que funcionou

Usar o algoritmo greedy para este caso tornou-se eficiente pois, face à restrição temporal de 5 minutos, a resolução em cerca de 17 segundos, mantendo a consistência em 100% e a cobertura em 98,7%, excedeu as expectativas.

A separação estrita entre computação Python e geração de linguagem natural revelou-se igualmente acertada, garantindo que os números no relatório final são rastreáveis e verificáveis, com uma taxa de alucinação de apenas 4,3%. No pipeline analítico, a adaptação à realidade física da loja, contando apenas as zonas visitadas antes da caixa e o preenchimento com zeros na deteção de anomalias revelaram-se cruciais para extrair sentido lógico dos dados.

Na integração com o LLM, a estratégia few-shot com zonas fictícias nos exemplos superou a estratégia zero-shot em acionabilidade, reduzindo também a tendência do modelo para copiar os exemplos dados.

6.2 O que não funcionou

O funil de clientes manteve-se invertido apesar das várias abordagens testadas como, por exemplo, a exclusão de trajetórias incompletas. Isto deve-se, e como referido várias vezes ao longo deste relatório, à planta da loja, que permite ir diretamente da entrada à caixa. O modelo LLM produziu também recomendações ocasionalmente genéricas, aquém do esperado para um sistema de inteligência de retalho.

6.3 O que faria de diferente

Se houvesse oportunidade para alterar aspetos deste trabalho, faria o seguinte:

- **Stitching híbrido com janela deslizante:** em vez de um algoritmo greedy puro, implementar uma abordagem de janela deslizante. O algoritmo continuaria a processar os dados sequencialmente para manter a rapidez, mas aplicaria uma otimização global (por exemplo, o Algoritmo Húngaro) em pequenos blocos de 10 a 20 segundos. Isto resolveria as ambiguidades de cruzamento de clientes sem exceder o limite de 5 minutos.
- **Modelo LLM de maior dimensão:** utilizar um modelo como o llama3.1:70b ou um modelo via API para obter recomendações mais sofisticadas e contextualmente relevantes, reduzindo os casos de recomendações genéricas observados com o modelo de 8B parâmetros.
- **Re-identificação por atributos visuais:** incorporar uma nova camada de dados como, por exemplo, a cor da roupa, no processamento das câmaras para enriquecer o cálculo de associação. Isto permitiria contornar as falhas mecânicas dos sensores de entrada, associando com maior confiança as pessoas que geram trajetórias sem fim de uma única zona.

- **Contexto histórico no LLM:** enriquecer o contexto fornecido ao modelo com dados de semanas anteriores, permitindo comparações temporais mais ricas nos insights gerados.

A Anexos

A.1 Estratégia A (Zero-Shot) - prompt

System Prompt

És um consultor especialista em retalho e análise de comportamento de clientes em loja. A tua função é analisar métricas de uma loja de retalho e gerar insights acionáveis em português europeu para o gestor de loja. Respondes SEMPRE e APENAS com JSON válido, sem texto adicional antes ou depois.

User Prompt

Analisa as seguintes métricas de uma loja de retalho e gera exatamente 6 insights acionáveis.

{metrics_text}

Responde APENAS com JSON válido seguindo este schema exato:

```
{
  "insights": [
    {
      "id": "INS_001",
      "categoria": "trafego|zona|funil|anomalia|demografico",
      "titulo": "frase curta que resume o insight",
      "observacao": "o que os dados mostram: factos e números concretos",
      "implicacao": "o que isto significa operacionalmente",
      "recomendacao": "ação concreta que o gestor pode tomar",
      "urgencia": "imediatesta_semana|proximo_mes",
      "confianca": 0.0
    }
  ],
  "resumo_executivo": "3 bullets com os insights mais importantes"
}
```

Regras:

- Cada insight deve ter números concretos retirados dos dados
- As recomendações devem ser específicas e executáveis
- A confiança deve ser entre 0.0 e 1.0
- Responde em português europeu
- Não incluas texto fora do JSON

A.2 Estratégia B (Few-Shot) - prompt

System Prompt

És um consultor especialista em retalho e análise de comportamento de clientes em loja. A tua função é analisar métricas de uma loja de retalho e gerar insights acionáveis em português europeu para o

gestor de loja. Respondes SEMPRE e APENAS com JSON válido, sem texto adicional antes ou depois.

User Prompt

Analisa as seguintes métricas de uma loja de retalho e gera exatamente 6 insights acionáveis.

EXEMPLOS DE INSIGHTS (dados fictícios | apenas para ilustrar o formato)

MAU INSIGHT (genérico, vago | NÃO fazer assim):

```
{
  "titulo": "A zona de frescos teve bastante tráfego",
  "observacao": "Muitas pessoas visitaram esta zona",
  "implicacao": "É uma zona popular",
  "recomendacao": "Melhorar o atendimento ao cliente"
}
```

BOM INSIGHT (específico, com números, acionável | fazer assim):

```
{
  "titulo": "Z_X9 (vinhos) tem dwell time 3x acima da média da loja",
  "observacao": "Z_X9 registou dwell médio de 187s vs média de 62s nas restantes secções; taxa de paragem de 0.91 com 312 visitantes",
  "implicacao": "Clientes passam muito tempo a decidir | secção com alto potencial de venda assistida",
  "recomendacao": "Colocar promotor em Z_X9 às sextas e sábados entre 16h-19h quando o tráfego é 45% acima da média semanal"
}
```

MAU INSIGHT (anomalia vaga | NÃO fazer assim):

```
{
  "titulo": "Houve uma anomalia na loja",
  "observacao": "O tráfego foi diferente do normal",
  "recomendacao": "Investigar o que aconteceu"
}
```

BOM INSIGHT (anomalia específica | fazer assim):

```
{
  "titulo": "Pico anómalo em Z_M3 na terça às 11h",
  "observacao": "Z_M3 teve 89 visitantes na terça às 11h vs média de 31 nos mesmos dias/horas anteriores (z=+4.1, acima da média)",
  "implicacao": "Possível evento promocional não registado ou redirecionamento de fluxo por obra noutro corredor",
  "recomendacao": "Cruzar com registos de promoções e obras da terça-feira; se recorrente, reforçar stock em Z_M3 antes das 11h"
}
```

FIM DOS EXEMPLOS | usa APENAS os dados reais fornecidos abaixo

IMPORTANTE: Os exemplos acima são fictícios. Gera insights NOVOS

baseados EXCLUSIVAMENTE nos dados abaixo.

DADOS DA SEMANA:

{metrics_text}

Responde APENAS com JSON válido seguindo este schema exato, com UMA categoria por insight:

```
{
  "insights": [ ... ],
  "resumo_executivo": "3 bullets com os insights mais importantes"
}
```

Regras:

- Cada insight deve citar números específicos dos dados acima
- Cada insight deve ter exatamente UMA categoria
- As recomendações devem ser concretas e executáveis
- A confiança deve ser entre 0.0 e 1.0
- Responde em português europeu
- Não incluas texto fora do JSON

A.3 Relatório semanal - prompt

System Prompt

És um consultor especialista em retalho. Escreves briefings semanais para gestores de loja sem formação técnica, em português europeu, com linguagem direta e números concretos. Nunca uses termos como 'z-score', 'dataset' ou 'pipeline'.

REGRA FUNDAMENTAL: Usa APENAS os números fornecidos. Nunca inventes ou estimes valores.

User Prompt

Escreve o briefing semanal completo da loja para a semana de {week_start} a {week_end}.

{metrics_ctx}
{insights_ctx}

Escreve em Markdown com EXATAMENTE estas 6 secções:

1. Resumo Executivo (máx. 150 palavras)
2. Performance de Tráfego
3. Análise de Zonas
4. Funil de Clientes
5. Anomalias da Semana (sem usar 'z-score')
6. Recomendações para a Próxima Semana (máx. 5 ações)

REGRAS: Usa APENAS os números fornecidos. Português europeu. Sem termos técnicos.

A.4 Excerto de metrics.json

```
{
  "traffic": {
    "total_visitors_week": 14390,
    "busiest_day": { "day_name": "sábado", "visitors": 2550 },
    "quietest_day": { "day_name": "terça", "visitors": 1620 },
    "peak_hour": 17,
    "avg_visit_duration_min": 2.3
  },
  "funnel": {
    "total_visitors": 10712,
    "reached_navigation": { "count": 1216, "pct": 11.4 },
    "reached_sections": { "count": 596, "pct": 5.6 },
    "reached_checkout": { "count": 3296, "pct": 30.8 },
    "completed_purchase": { "count": 2000, "pct": 18.7 }
  },
  "anomalies": {
    "total_anomalies": 22,
    "anomalies": [
      {
        "zone_id": "Z_N6",
        "hour": 14,
        "visitors_d7": 50,
        "baseline_mean": 24.3,
        "z_score": 3.79,
        "direction": "acima"
      }
    ]
  }
}
```

A.5 Excerto de insights.json — Estratégia B

```
{
  "strategy_B_few_shot": {
    "insights": [
      {
        "id": "INS_001",
        "categoria": "zona",
        "titulo": "Z_CK tem dwell time 4x acima da média da loja",
        "observacao": "Z_CK registou dwell médio de 89.12s vs média de 22.3s nas restantes secções; taxa de paragem de 0.99 com 4799 visitantes na semana",
        "implicacao": "Clientes passam muito tempo na saída | zona com alto potencial de venda assistida",
        "recomendacao": "Colocar promotor em Z_CK às sextas e sábados entre 16h-19h",
        "urgencia": "esta_semana",
      }
    ]
  }
}
```

```

    "confianca": 0.9
  },
  {
    "id": "INS_004",
    "categoria": "anomalia",
    "titulo": "Pico anómalo em Z_N6 na quarta às 14h",
    "observacao": "Z_N6 teve 50 visitantes na quarta às 14h
      vs média de 24.3 nos mesmos dias/horas anteriores
      (z=+3.79, acima da média)",
    "implicacao": "Possível evento promocional não registado",
    "recomendacao": "Cruzar com registos de promoções da
      quarta-feira; reforçar stock em Z_N6 antes das 14h",
    "urgencia": "esta_semana",
    "confianca": 0.9
  }
]
},
"comparison": {
  "strategy_A_zero_shot": {
    "avg_specificity": 0.53,
    "avg_actionability": 0.77,
    "avg_total": 0.77
  },
  "strategy_B_few_shot": {
    "avg_specificity": 0.90,
    "avg_actionability": 0.77,
    "avg_total": 0.88
  }
}
}
}

```

A.6 Relatório semanal gerado

Briefing Semanal | 2025-03-10 a 2025-03-16

1. Resumo Executivo

O sábado foi o dia mais movimentado da semana, com 2550 visitantes. A zona Z_CK tem um tempo de permanência 4x acima da média da loja.

2. Performance de Tráfego

Total de visitantes na semana: 14390.
 Dia mais movimentado: sábado com 2550 visitantes.
 Dia mais calmo: terça-feira com 1620 visitantes.
 Hora de pico: 17h. Duração média de visita: 2.3 minutos.

3. Análise de Zonas

Z_CK tem tempo de permanência muito elevado | recomenda-se colocar promotor às sextas e sábados entre 16h-19h.

4. Funil de Clientes

Chegaram a corredores: 1216 (11,4%).

Chegaram a secções: 596 (5,6%).

Chegaram à caixa: 3296 (30,8%).

Completaram compra: 2000 (18,7%).

5. Anomalias da Semana

Pico em Z_N6 na quarta às 14h: 50 visitantes vs média de 24,3.

6. Recomendações para a Próxima Semana

1. Colocar promotor em Z_CK às sextas e sábados entre 16h-19h.
2. Revisar layout de Z_E1 para maximizar oportunidades de venda.
3. Reforçar recursos no sábado | dia mais movimentado da semana.
4. Investigar pico de Z_N6 às quartas às 14h.
5. Melhorar navegação interior para aumentar tráfego nas secções.